

3. Discuss the Preparation Learning Activity

What part of the learning activity was the hardest for you?

The hardest part for me was understanding when to use `override`. I had to remember that the base class defines the method and the derived classes provide their own versions. The Shapes activity was really preparing us for Eternal Quest. Instead of Shape, our base class is Goal. Instead of Square, Rectangle, and Circle, we'll have SimpleGoal, EternalGoal, and ChecklistGoal. The inheritance pattern is almost the same.

4. Review the Program Specification

We are asked to gamify tracking different kinds of goals.

- 1. Simple goals that can be marked complete and user gain value/reward**
- 2. Eternal goals that are never complete, but the user gets some value each time they get recorded. Ex. reading scriptures**
- 3. Goal checklist that must be accomplished a certain # of times to be complete. Ex. every time you accomplish that goal you get 10pts but by the 10th time you get 100pt bonus.**
- 4. Display user's score**
- 5. Allow user to create new goals of any type.**
- 6. Allow user record an event (accomplishing a goal and receive points for it).**
- 7. Show a list of goals. It should display whether the goal has been completed or not , for checklist goals it should show how many times the goal has been completed (ex. $\frac{2}{3}$ times).**
- 8. Allow the user's goals & current score to be saved and loaded.**

What does the program do?

Tracks goals and awards points when goals are completed.

What user inputs does it have?

Examples:

Goal name

Goal description

Point value

Goal type

Checklist target

Checklist bonus
Menu selections
File names for save/load

What output does it produce?

Examples:
Current score
Goal list
Goal completion status
Messages when points are earned

How does the program end?

User chooses the Exit option from the menu.

5. Determine the classes

What are good candidates for classes in this program? What are the primary responsibilities of each class?

- ❖ Goal - Stores common information:
 - Name
 - Description
 - Points
- ❖ SimpleGoal - Can be completed once.
- ❖ EternalGoal - Never completed.
- ❖ ChecklistGoal - Completed after reaching a target.

While it is possible to handle all of the menu system and its related functions, including the saving of points, as static functions in the Program.cs file, because there is enough complexity there, it would be nice to encapsulate all that functionality into a separate "manager" class.

- ❖ GoalManager - Handles:
 - Menus
 - Goal list
 - Score
 - Save/load

What is a benefit to having a separate class like `GoalManager` instead of just using static functions in `Program.cs` for all of the menu-related functionality?

Keeps code organized, `Program.cs` stays small, it'll be easier to maintain and it follows encapsulation

6. Define class behaviors

What are the behaviors this class will have in order to fulfill its responsibilities? (In other words, what things should this class *do*?)

Goal methods:

- `RecordEvent()` - This method should do whatever is necessary for each specific kind of goal, such as marking a simple goal complete and adding to the number of times a checklist goal has been completed. It should return the point value associated with recording the event (keep in mind that it may contain a bonus in some cases if a checklist goal was just finished, for example).
- `IsComplete()` - This method should return true if the goal is completed. The way you determine if a goal is complete is different for each type of goal.

In addition to recording the event and checking if it is complete. Each goal will need to be shown in a list (with the checkbox etc), and also each one will need a string representation that can be saved to a text file. These should also be declared in the base class and overridden in the derived classes as needed:

- `GetDetailsString()` - This method should return the details of a goal that could be shown in a list. It should include the checkbox, the short name, and description. Then in the case of the `ChecklistGoal` class, it should be overridden to shown the number of times the goal has been accomplished so far.
- `GetStringRepresentation()` - This method should provide all of the details of a goal in a way that is easy to save to a file, and then load later.

- **Goal**
 - **RecordEvent()** : **void** (*Abstract*)
 - **IsComplete()** : **bool** (*Abstract*)
 - **GetDetailsString()** : **string**
 - **GetStringRepresentation()** : **string** (*Abstract*)
- **SimpleGoal**
 - **RecordEvent()** : **void** (*Override*)
 - **IsComplete()** : **bool** (*Override*)
 - **GetStringRepresentation()** : **string** (*Override*)
- **EternalGoal**
 - **RecordEvent()** : **void** (*Override*)
 - **IsComplete()** : **bool** (*Override*)
 - **GetStringRepresentation()** : **string** (*Override*)
- **ChecklistGoal**
 - **RecordEvent()** : **void** (*Override*)
 - **IsComplete()** : **bool** (*Override*)
 - **GetDetailsString()** : **string** (*Override*)
 - **GetStringRepresentation()** : **string** (*Override*)

In addition to these behaviors for the goals, the goal manager class will need behaviors related to the menu system such as the following:

- **Start** - This is the "main" function for this class. It is called by Program.cs, and then runs the menu loop.
- **DisplayPlayerInfo** - Displays the players current score.
- **ListGoalNames** - Lists the names of each of the goals.
- **ListGoalDetails** - Lists the details of each goal (including the checkbox of whether it is complete).
- **CreateGoal** - Asks the user for the information about a new goal. Then, creates the goal and adds it to the list.
- **RecordEvent** - Asks the user which goal they have done and then records the event by calling the **RecordEvent** method on that goal.
- **SaveGoals** - Saves the list of goals to a file.
- **LoadGoals** - Loads the list of goals from a file.

What is a benefit of making the `RecordEvent` method an abstract method instead of leaving it blank or providing a default version?

Because every goal records progress differently. Simple goals- marks complete, eternal goal -gives points every time, checklist goal - increases count and may award a bonus.

Notice that in the design above, the `GetDetailsString` method has a default implementation and is only overridden by the `ChecklistGoal` class. Why is the `GetDetailsString` method only overridden by the `ChecklistGoal` class and not the other two derived classes?

Checklist goals need extra information. Ex. Read Scriptures Completed 4/10 times . Simple & eternal goals don't need that extra display.

7. Define class attributes

What attributes does this class need to fulfill its behaviors? (In other words, what variables should this class store?) What are the data types of these member variables?

- `Goal`
 - `_shortName : string`
 - `_description : string`
 - `_points : int`
- `SimpleGoal`
 - `_isComplete : bool`
- `EternalGoal`
 - *None needed*
- `ChecklistGoal`
 - `_amountCompleted : int`
 - `_target : int`
 - `_bonus : int`

In addition to the goal classes, the **GoalManager** class will need to keep track of the list of goals as well as the players score.

- **GoalManager**
 - `_goals : List<Goal>`
 - `_score : int`

Notice that the **EternalGoal class does not define any member variables. What is a reason to still have this class, even though it does not define any member variables?**

Because it has a different behavior. Its RecordEvent and IsComplete work differently. Behavior can justify a class even when it adds no new data.

8. Define Constructors

What constructors should each class have? In other words, what parameters should you pass in when creating an object of that type.

The following constructors are appropriate for these classes:

- `Goal(name : string, description : string, points : int)`
- `SimpleGoal(name : string, description : string, points : int)` - This should also set the the variables for if it is complete to be false.
- `EternalGoal(name : string, description : string, points : int)`
- `CheckListGoal(name : string, description : string, points : int, target : int, bonus : int)` - In addition to the standard attributes, a checklist goal also needs the target and the bonus amounts. Then, it should set the amount completed to begin at 0.
- `GoalManager()` - Initializes an empty list of goals and sets the player's score to be 0.

What other work needs to be done to set up these objects? For example, does the constructor need to run code to perform set up tasks, like creating lists, iterating through variables, etc.

For most of the goal classes, the constructor mainly needs to initialize the member variables with the values passed in and set any default values.

Ex:

- **SimpleGoal**
 - Set `_isComplete` to `false` because a new goal starts incomplete.
- **ChecklistGoal**
 - Set `_amountCompleted` to `0`.
 - Store the target number and bonus amount.
- **EternalGoal**
 - Doesn't require any additional setup beyond the common goal information.
- **GoalManager**
 - Create an empty `List<Goal>` to store goals.
 - Set the player's score to `0`.

Why does the `SimpleGoal` constructor not need a parameter for the `_isComplete` variable?

It doesn't need it because every new goal starts incomplete. The constructor simply set: `_isComplete = false;`

9. Review the Design

